Lecture 14
Operating System Security

Prof. Yinqian Zhang

Spring 2025

Outline

- Confinement
  - Isolation
  - Access control
  - Trusted execution environment
- System Hardening
  - Vulnerability discovery: Program analysis, Fuzzing
  - Exploit prevention: ASRL, Stack Canary, CFI
- Detection
  - Malware detection
  - Host-based intrusion detection

Outline

- **Confinement**
  - Isolation
  - Access control
  - Trusted execution environment
- System Hardening
  - Vulnerability discovery: Program analysis, Fuzzing
  - Exploit prevention: ASRL, Stack Canary, CFI
- Detection
  - Malware detection
  - Host-based intrusion detection

Run Untrusted Code

- System must run untrusted code
  - Programs from untrusted Internet sites:
    - Mobile apps, Javascript, browser extensions, etc.
  - Applications exposed to the external data:
    - Web servers, browsers, pdf viewer, etc.
  - Kernel components downloaded from third-party sources:
    - Device drivers, user-defined kernel programs
- System must "confine" it
  - Prevent it from accessing memory/files of other applications
  - Prevent it from doing things only kernel can do
    - Privileged instructions, I/O, privileged registers, etc.
  - Prevent its faults from interfering with the rest system

Confinement

- Confinement is a principle in computer science and software engineering that aims to
  - restrict the interactions of a program to a limited set of resources
  - prevent unintended interference with other parts of the system
- Key approaches to achieve confinement
  - **Isolation**: boundary enforcement
    - Ensures that components operate independently to prevent interference or failure propagation
  - **Access control**: permission management
    - Governs who can perform what actions on which resources

Outline

- Confinement
  - **Isolation**
  - Access control
  - Trusted execution environment
- System Hardening
  - Vulnerability discovery: Program analysis, Fuzzing
  - Exploit prevention: ASRL, Stack Canary, CFI
- Detection
  - Malware detection
  - Host-based intrusion detection

Isolation

- Physical isolation
  - Run isolated programs on a **physically separated** computer
- Hardware isolation
  - Run isolated program in **lower privilege levels**, e.g., user mode
  - Run isolated program **within a process**
    - Intel Memory Protection Keys (MPK)
    - ARM Memory Tagging Extension (MTE)
- Software isolation
  - Run isolated program in a confined environment within
    - Kernel: eBPF
    - User process: Software fault isolation (SFI), WebAssembly

Isolation between Privilege Levels

- Recall: dual-mode operations
  - x86: ring 3 vs ring 0
  - RISC-V: U-mode vs S-mode
  - Arm: EL0 vs EL1
- Recall: How are user programs Isolated?
  - Executing privileged instructions or accessing privileged registers:
    - Denied by CPU and exceptions raised: CSRs, I/O instructions
  - Page tables configured by OS kernel:
    - Memory of other processes not mapped in the page table
  - U-bit in PTEs:
    - Restrict accesses to kernel pages

Isolation within Privilege Levels

- Intra-process compartmentalization
  - Software-based solutions
    - Software fault isolation (SFI)
  - Hardware-based solutions
    - Intel MPK
    - Arm MTE
- Intra-kernel compartmentalization
  - Extended Berkeley Packet Filter (eBPF)

Software fault isolation (SFI)

- SFI enforces security boundaries between software components at runtime via compile-time or runtime instrumentation
- Goals:
  - Safety: Isolate untrusted or error-prone modules.
  - Portability: Work without hardware support.
- Take aways
  - Fine-grained isolation without hardware support
  - Incurs 5% ~ 20% performance overhead
- Example: Google Native Client (NaCl) runs untrusted x86 code safely in browsers.
  - All indirect jumps must be to aligned 32-byte boundaries.
  - Data accesses masked to a 4GB sandbox.

SFI in Action

- Memory Sandboxing
  - Restricts memory accesses to predefined regions
- Control-Flow Restriction
  - Ensures jumps/calls only target valid entry points
  - Example: Indirect branches must land on aligned, whitelisted addresses
- Instruction-Level Sandboxing
  - Filters dangerous instructions (e.g., syscall, int 0x80)

```
; ;Before SFI: Unsafe indirect jump
jmp *%eax  ; Could jump anywhere

; After SFI: Sandboxed jump
andl $0xFFFFFFE0, %eax  ; Force 32-byte alignment
jmp *%eax               ; Now jumps only to allowed targets
```

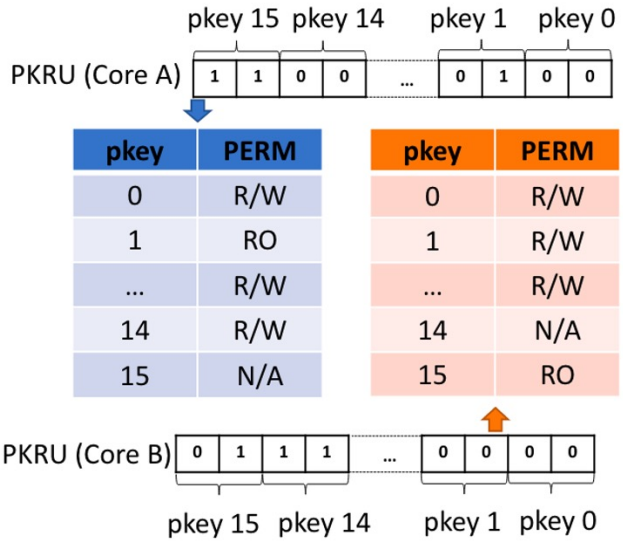
Intel Memory Protection Keys (MPK)

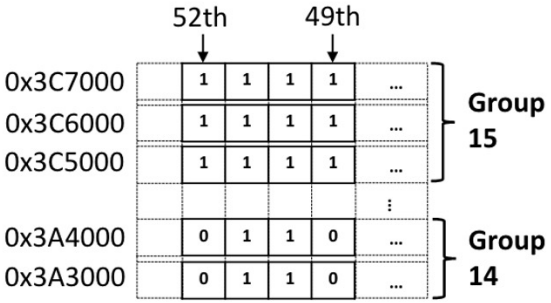
- Intel MPK assigns a protection key to groups of memory pages and controls access rights via a protection key rights register
- Key components:
  - PKRU Register: A 32-bit register for user-mode protection keys, controlling access rights for each key.
  - RDPKRU/WRPKRU Instructions: Instructions to read from and write to the PKRU register
- Benefits:
  - Changing access rights of a page takes less than 20 CPU cycles
  - Allows changing permissions for up to 16 different page groups simultaneously

MPK in Action

- Protection Key:
  - A 4-bit value stored in PTEs, ranging from 0 to 15.
  - Acts as an index to the protection key rights register.
- Each key has two bits in the PKRU register:
  - Access Disable (AD) and Write Disable (WD)
  - (0, 0): Read and Write allowed
  - (1, 0): Read allowed, Write denied
  - (0, 1): Read denied, Write denied
  - (1, 1): Read denied, Write denied

- Instructions:
  - WRPKRU: Writes the value of EAX into the PKRU register.
  - RDPKRU: Reads the value of the PKRU register into EAX





Extended Berkeley Packet Filter (eBPF)

- eBPF is a **safe, efficient** virtual machine in the Linux kernel.
  - Runs user-defined programs without modifying kernel source or loading modules.
  - Evolved from the classic BPF (Berkeley Packet Filter), a network packet filtering technology created in the 1990s.
- Use Cases:
  - Networking: XDP (high-speed packet filtering).
  - Security: Detect malicious syscalls (Falco).
  - Tracing: Profile applications (BCC, bpftrace).

eBPF in Action

- How It Works:
  - Load: User space loads eBPF bytecode into kernel.
  - Verify: Kernel checks safety (no loops, valid memory access).
  - Attach: Program hooks to events (syscalls, packets).
  - Run: JIT-compiled execution when events trigger.

```
// Define a BPF map to store the count of system calls
struct bpf_map_def SEC("maps") syscall_count = {
    .type = BPF_MAP_TYPE_HASH,
    .key_size = sizeof(u32),
    .value_size = sizeof(u64),
    .max_entries = 1024,
};

// BPF program to count the number of times a specific syscall is invoked
SEC("tracepoint/syscalls/sys_enter_openat")
int count_syscall(struct pt_regs *ctx) {
    u32 pid = bpf_get_current_pid_tgid() >> 32;
    u64 *value;

  // Increment the count for the current PID
    value = bpf_map_lookup_elem(&syscall_count, &pid);
    if (value) {
        (*value)++;
    } else {
        u64 init_val = 0;
        bpf_map_update_elem(&syscall_count, &pid, &init_val, BPF_ANY);
    }
```

Outline

- Confinement
  - Isolation
  - **Access control**
  - Trusted execution environment
- System Hardening
  - Vulnerability discovery: Program analysis, Fuzzing
  - Exploit prevention: ASRL, Stack Canary, CFI
- Detection
  - Malware detection
  - Host-based intrusion detection

Access Control

- Access control restricts unauthorized users and systems from accessing resources
- Key components: subjects, objects, and permissions
  - Subject: users/processes
  - Objects: files/devices
  - Permissions: read/write/execute
- Implementations of access control
  - Access control matrix
  - Access control list
  - Capability

Implementation of Access Control

- Access control matrix
- Access control list
  - An access control list (ACL) is attached to each object
  - <subject, permissions>, …, <subject, permissions>
- Capability
  - A list of tokens held by subjects

	File 1	File 2	File 3	Device 1	Device 2
Process 1	read	read/write			
Process 2	execute		read/write		read/write
User 1		read/write	read		
User 2	execute			read/write	read

Examples of Linux Access Control

- Discretionary Access Control (DAC):
  - Access control based on the identity of the user or process and the permissions assigned to them.
  - Permissions are set by the owner of the resource.
  - Typically provides user-level or group-level control.
  - Commonly implemented through file permissions (e.g., chmod, chown)
- Mandatory Access Control (MAC):
  - Access control based on security labels or classifications assigned to subjects and objects.
  - Access is controlled by a central authority through security policies.
  - Provides fine-grained control based on security labels.
  - Implemented through security frameworks (e.g., SELinux, AppArmor).

Capabilities

- A capability is a **secure**, **unforgeable** token/key that grants a subject specific permissions to access an object in a system.
- Unlike ACL, capabilities are **held by subjects**.
- Workflows of capabilities
  - **Issuance**: A trusted authority (e.g., OS/kernel) grants a capability to a subject.
  - **Access Request**: The subject presents the capability to access an object.
  - **Validation**: The system verifies the capability's authenticity (e.g., via cryptography).
  - **Action**: Permissions in the capability are enforced (e.g., read-only).

Key Properties of Capabilities

- Unforgeable
  - Cannot be tampered with (e.g., cryptographically signed or hardware-enforced).
- Delegable
  - Can be shared or revoked dynamically.
- Self-Describing
  - Contains both the object reference and permissions (e.g., "Read file X").
- No Global Checks
  - Permissions are checked locally (via the capability), not in a central ACL.

Examples: HarmonyOS

- HarmonyOS integrates **fine-grained capabilities** to manage permissions dynamically.
- Example: Camera Access in HarmonyOS
- Step 1: App Requests Capability
- Step 2: User Approves/Denies
  - A system dialog pops up: "Allow AppX to use the camera?"
- Step 3: TEE Validates Token
  - The request is cryptographically verified in the TEE before access is granted.
- Step 4: Access Revocation
  - Users can revoke capabilities anytime in Settings → App Permissions.

```
{//example of a capability token
 "app_id": "com.example.app", "capability": "ACCESS_CAMERA",
 "expiry": "2025-12-31", "signature": "SHA3-256_HMAC"}
```

Comparison of ACL and Capability

FEATURE	ACL	CAPABILITY
Storage	Attached to the *object* (e.g., file).	Held by the *subject* (e.g., token).
Scalability	Slower (checks centralized ACLs).	Faster (local checks).
Delegation	Harder (modify ACLs per user).	Easier (share capabilities).
Example	Linux file permissions.	OAuth tokens, AWS IAM keys.

Outline

- Confinement
  - Isolation
  - Access control
  - **Trusted execution environment**
- System Hardening
  - Vulnerability discovery: Program analysis, Fuzzing
  - Exploit prevention: ASRL, Stack Canary, CFI
- Detection
  - Malware detection
  - Host-based intrusion detection

Trusted Execution Environment (TEE)

- Trusted execution environment (TEE) is a security feature of modern commodity CPUs
  - TEEs are hardware/software extensions of CPUs
  - It enables multiple security domains to share CPU, memory and I/O
- Most modern commodity CPUs have TEEs
  - Intel SGX and TDX
  - Arm TrustZone and CCA
  - AMD SEV
- TEE shields sensitive applications from untrusted OS/VMM

Enabling Techniques of TEEs

- CPU isolation
  - The external software of TEE (including application software and system software) cannot access the confidential data inside TEE-protected environment, including memory, registers, CPU cache
- Memory encryption
  - The physical memory used by TEE is encrypted on-the-fly by a hardware encryption engine. Encryption keys are dynamically generated and managed by the TEE hardware.
- Remote attestation
  - Users can ensure the integrity of the initial state of the software running in TEE through remote attestation, which relies on the hardware root of trust on the CPU and the trust infrastructure supported by the CPU manufacturer.

Outline

- Confinement
  - Isolation
  - Access control
  - Trusted execution environment
- **System Hardening**
  - Vulnerability discovery: Program analysis, Fuzzing
  - Exploit prevention: ASRL, Stack Canary, CFI
- Detection
  - Malware detection
  - Host-based intrusion detection

System Hardening

- Vulnerability discovery
  - Goals: Discover bugs/vulnerabilities in software, **before** they are deployed in the system
  - Methods: program analysis & fuzz testing (fuzzing)
  - Source code analysis or binary analysis
- Exploit prevention
  - Goals: Prevent vulnerable software from successful exploitation, **after** they are deployed in the system
  - Methods: information hiding, control flow enforcement

Outline

- Confinement
  - Isolation
  - Access control
  - Trusted execution environment
- System Hardening
  - **Vulnerability discovery: Program analysis, Fuzzing**
  - Exploit prevention: ASRL, Stack Canary, CFI
- Detection
  - Malware detection
  - Host-based intrusion detection

Vulnerability Discovery Techniques

- **Static Analysis:** Analyze source code without execution
  - data flow analysis: Cppcheck
  - Static taint analysis: FlowDroid, Joern
  - Symbolic execution: Angr, SAGE
- **Dynamic Analysis:** test running applications
  - Fuzzing: AFL++, libFuzzer
  - Dynamic taint analysis: TaintCheck, TaintDroid
- **Hybrid Approaches**
  - Concolic Execution: Combine symbolic + concrete execution
  - Tools: KLEE, QSYM

Fuzzing

- Fuzzing is a technique to discover bugs by injecting random, malformed, or unexpected inputs into software.
- It crashes programs to uncover **vulnerabilities** (e.g., buffer overflows, memory leaks).
- How does it work:
  - **Input Generation**: generate/mutate input from random seeds
  - **Execution**: Feed inputs to target (e.g., API, OS kernel).
  - **Monitoring**: Detect crashes/timeouts.

Fuzzing Techniques and Tools

- **Input Generation Strategy**
  - Mutation-Based: Modifies existing inputs (e.g., bit-flipping)
  - Generation-Based: Creates inputs from formal specifications
- **Feedback Mechanism**
  - Coverage-Guided: Uses execution traces (e.g., branch coverage)
  - Black-Box: No program feedback
  - Grey-Box: Lightweight instrumentation
- **Target Knowledge**
  - White-Box: Uses program analysis (concolic execution)
  - Black-Box: No internal knowledge
  - Hybrid: Combines static/dynamic analysis

Outline

- Confinement
  - Isolation
  - Access control
  - Trusted execution environment
- System Hardening
  - Vulnerability discovery: Program analysis, Fuzzing
  - **Exploit prevention: ASRL, Stack Canary, CFI**
- Detection
  - Malware detection
  - Host-based intrusion detection

Address Space Layout Randomization (ASLR)

- ASLR is a security technique that randomizes the virtual addresses of system and application components (e.g., stack, heap, libraries).
- **Purpose of ASLR**
  - Mitigates memory corruption exploits (buffer overflows, ROP, jump-to-libc).
  - Forces attackers to guess memory addresses (reduces exploit reliability).
- **ASLR Implementation**
  - Windows: Enabled by default since Windows Vista.
  - Linux: Full randomization since kernel 3.14
  - macOS: Strict ASLR w/ additional entropy.
- ASLR makes exploits harder but not impossible, it is critical for modern exploit mitigations (combined with DEP, CFI, Stack Canaries).

Stack Canary

- A runtime security mechanism that detects buffer overflows by placing a random value between the stack variables and the return address
  - Compiler-Inserted Guard: A random random value is placed before the return address.
  - Overflow Detection: If the canary is modified (due to a buffer overflow), the program terminates.
- Stack canary prevents stack-based buffer overflow exploits and mitigates ROP by stopping stack corruption.
- Stack canary does not prevent heap overflows or data-only attacks; Bypassable if the attacker can leak or guess the canary.

Control Flow Integrity (CFI)

- CFI is a security mechanism that **restricts program execution** to valid control-flow paths, preventing exploits like:
  - ROP (Return-Oriented Programming)
  - JOP (Jump-Oriented Programming)
  - Call/return hijacking

Method	Implementation	Example
Forward-Edge CFI	Protects indirect calls/jumps	Clang CFI, Intel CET (IBT)
Backward-Edge CFI	Protects function returns	Shadow Stack (Intel CET)
Fine-Grained CFI	Strict call-target validation	Microsoft CFG, ARM BTI
Coarse-Grained CFI	Basic block-level enforcement	Early academic prototypes

CFI: Effectiveness and Challenges

- CFI is critical for modern exploit mitigation but requires hardware-software co-design
- Hardware Support
  - **Intel CET** (Control-Flow Enforcement Technology):
    - Shadow Stack for backward-edge
    - Indirect Branch Tracking (IBT) for forward-edge
  - **ARM BTI** (Branch Target Identification):
    - Marks valid branch targets
- Modern CFI stops about 90% of ROP attacks with 1–15% runtime performance cost

Outline

- Confinement
  - Isolation
  - Access control
  - Trusted execution environment
- System Hardening
  - Vulnerability discovery: Program analysis, Fuzzing
  - Exploit prevention: ASRL, Stack Canary, CFI
- **Detection**
  - **Malware detection**
  - Host-based intrusion detection

Malware Detection

- Goals: identify and mitigate malicious software (malware) that can harm systems, steal data, or disrupt operations.
- Types of malware
  - Viruses: Self-replicating programs that infect files.
  - Worms: Self-propagating programs that spread over networks.
  - Trojans: Malicious programs disguised as legitimate software.
  - Ransomware: Encrypts data and demands payment for decryption.
  - Spyware: Software that collects information without user consent.
  - Adware: Displays unwanted advertisements.
  - Rootkits: Malware that gains root-level access and hides its presence.

Malware Detection Techniques

- **Signature-Based Detection**: Identifies known malware by matching patterns or signatures.
- **Heuristic-Based Detection**: Analyzes behavior to detect unknown or new malware.
- **Behavioral Analysis**: Monitors real-time activities to identify malicious behavior.
- **Machine Learning-Based Detection**: Uses algorithms to learn and detect patterns indicative of malware.
- **Sandboxing**: Runs suspicious files in a controlled environment to observe behavior.
- **Anomaly Detection**: Identifies deviations from normal system behavior.

Host-based Intrusion Detection (HIDS)

- Monitors and analyzes activities on a single host to detect potential intrusions or malicious activities.
- Key Components
  - **Event Sensors**: Collect data from various sources such as system logs, application logs, and network traffic.
  - **Event Analyzers**: Process and analyze the collected data to identify suspicious activities.
  - **Response Mechanisms**: Take appropriate actions in response to detected intrusions, such as alerting administrators or blocking malicious activities.

HIDS Techniques

- Signature-Based Detection: Identifies known threats by matching patterns or signatures against a database of known attack signatures.
- Anomaly-Based Detection: Detects unusual or unexpected behavior that deviates from a predefined baseline of normal activity.
- Behavioral Analysis: Monitors the behavior of applications and processes to identify malicious activities in real-time.
- Log Analysis: Examines system and application logs for signs of suspicious activities or potential security breaches.

Network Security in a Nutshell

Domain	Technologies/Techniques	Threats Mitigated
Perimeter Security	Firewalls, IDS/IPS, NAC	Unauthorized access, intrusions
Data Protection	TLS/SSL, VPNs, Data Loss Prevention (DLP)	Eavesdropping, man-in-the-middle
Access Control	Zero Trust, MFA, IAM	Credential theft, insider threats
Threat Monitoring	SIEM, NDR (Network Detection & Response)	APTs, lateral movement
Wireless Security	WPA3, 802.1X, Rogue AP detection	Wi-Fi spoofing, Evil Twin attacks
Cloud/Edge Security	SASE, CASB, Zero Trust Network Access (ZTNA)	Misconfigurations, cloud breaches

Applied Cryptography in a Nutshell

- **Data Confidentiality**
  - Encryption (AES, RSA, ChaCha20)
  - Secure channels (TLS, VPNs, SSH)
- **Data Integrity**
  - Hash functions (SHA-3, BLAKE3)
  - MACs (HMAC, Poly1305)
- **Authentication**
  - Digital signatures (ECDSA, EdDSA)
  - Zero-knowledge proofs

Domain	Technologies	Use Cases
Network Security	TLS, IPsec, WireGuard	Secure web browsing, VPNs
System Security	Full-disk encryption	Device protection
Blockchain	Elliptic curve, zk-SNARKs	Bitcoin, Zcash
Privacy Tech	Homomorphic encryption, DP	Secure data processing
Auth/Identity	OAuth, FIDO2, Passkeys	Passwordless auth

Thank you!

